

Gleam for Python users Hello productive pragmatic Pythonistas! a soft wavey boundary between two sections of the website Comments Variables Match operator Variables type annotations Functions Exporting functions Function type annotations Referencing functions Labelled arguments Operators Constants Blocks Data types Strings Tuples Lists Dicts Flow control Case Try Type aliases Custom types Records Unions Opaque custom types Modules Imports Named imports Unqualified imports

Comments Python In Python, comments are written with a # prefix. # Hello, Joe! A docstring (matching """) that occurs as the first statement in a module, function, class, or method definition will become the __doc__ attribute of that object. def a_function(): """Return some important data.""" pass

Gleam In Gleam, comments are written with a // prefix. // Hello, Joe! Comments starting with /// are used to document the following statement. Comments starting with //// are used to document the current module. /// This module is very important. /// The answer to life, the universe, and everything. const answer: Int = 42

Variables You can reassign variables in both languages. Python size = 50 size = size + 100 size = 1 Python has no specific variable keyword. You choose a name and that's it! Gleam Gleam has the let keyword before its variable names. let size = 50 let size = size + 100 let size = 1

Match operator Python Python supports basic, one directional destructuring (also called unpacking). Tuple of values can be unpacked and inner values can be assigned to left-hand variable names. (a, b) = (1, 2) # a == 1 # b == 2 # works also for for-loops for key, value in enumerate(a_dict): print(key, value)

Gleam In Gleam, let and = can be used for pattern matching, but you'll get compile errors if there's a type mismatch, and a runtime error if there's a value mismatch. For assertions, the equivalent let assert keyword is preferred. let #(x, _) = #(1, 2) let assert [] = [1] // runtime error let assert [y] = "Hello" // compile error, type mismatch

Variables type annotations Python Python is a dynamically typed language. Types are only checked at runtime and a variable can have different types in its lifetime. Type hints (Python 3+) are optional annotations that document the code with type information. These annotations are accessible at runtime via the __annotations__ module-level variable. These hints will mainly be used to inform static analysis tools like IDEs, linters... some_list: list[int] = [1, 2, 3]

Gleam In Gleam type annotations can optionally be given when binding variables. let some_list: List(Int) = [1, 2, 3] Gleam will check the type

annotation to ensure that it matches the type of the assigned value. It does not need annotations to type check your code, but you may find it useful to annotate variables to hint to the compiler that you want a specific type to be inferred.

Functions Python

In Python, you can define functions with the `def` keyword. In that case, the `return` keyword is mandatory.

```
def sum(x, y):  
    return x + y
```

Anonymous functions returning a single expression can also be defined with the `lambda` keyword and be assigned into variables.

```
mul = lambda x, y: x * y  
mul(1, 2)
```

Gleam

Gleam's functions are declared using a syntax similar to Rust or JavaScript. Gleam's anonymous functions have a similar syntax and don't need a `.` when called.

```
pub fn sum(x, y) { x + y }  
let mul = fn(x, y) { x * y }  
mul(1, 2)
```

Exporting functions Python

In Python, top level functions are exported by default. There is no notion of private module-level functions.

Gleam

In Gleam, functions are private by default and need the `pub` keyword to be public.

```
// this is public  
pub fn sum(x, y) { x + y }  
// this is private  
fn mul(x, y) { x * y }
```

Function type annotations Python

Type hints can be used to optionally annotate function arguments and return types. Discrepancies between type hints and actual values at runtime do not prevent interpretation of the code. Static code analysers (IDE tooling, type checkers like `mypy`) will be required to detect those errors.

```
def sum(x: int, y: int) -> int: return x + y  
def mul(x: int, y: int) -> bool: # no errors from the interpreter.  
    return x * y
```

Gleam

Functions can optionally have their argument and return types annotated in Gleam. These type annotations will always be checked by the compiler and throw a compilation error if not valid. The compiler will still type check your program using type inference if annotations are omitted.

```
pub fn add(x: Int, y: Int) -> Int { x + y }  
pub fn mul(x: Int, y: Int) -> Bool { // compile error, type mismatch x * y }
```

Referencing functions Python

As long as functions are in scope they can be assigned to a new variable. There is no special syntax to assign a module function to a variable.

Gleam

Gleam has a single namespace for value and functions within a module, so there is no need for a special syntax to assign a module function to a variable.

```
fn identity(x) { x }  
fn main() { let func = identity  
    func(100) }
```

Labelled arguments

Both Python and Gleam have ways to give arguments names and in any order.

Python

Keyword arguments are evaluated once at function definition time, and there is no evidence showing a noticeable performance penalty when using named arguments. When calling a function, arguments can be passed positionally, in the same order of the function declaration by name, in any order

```
def replace(inside: str,
```

each: str, with_string: str): pass # equivalent calls replace('hello world', 'world', 'you') replace(each='world', inside='hello world', with_string='you')

Gleam In Gleam arguments can be given a label as well as an internal name. Contrary to Python, the name used at the call-site does not have to match the name used for the variable inside the function.

```
pub fn
replace(inside string, each pattern, with replacement) { go(string, pattern,
replacement) } replace(each: ",", with: " ", inside: "A,B,C")
```

There is no performance cost to Gleam's labelled arguments as they are optimised to regular function calls at compile time, and all the arguments are fully type checked.

Operators

Operator Python Gleam

Notes

Equal == ==

In Gleam both values must be of the same type Strictly equal to == == Comparison in Gleam is always strict. (see note for Python) Reference equality is True only if the two objects have the same reference

Not equal != !=

In Gleam both values must be of the same type

Greater than > >

In Gleam both values must be ints

Greater than > >.

In Gleam both values must be floats

Greater or equal >= >=

In Gleam both values must be ints

Greater or equal >= >=.

In Gleam both values must be floats

Less than < <

In Gleam both values must be ints

Less than < <.

In Gleam both values must be floats

Less or equal <= <=

In Gleam both values must be ints

Less or equal <= <=.

In Gleam both values must be floats

Boolean and and &&

In Gleam both values must be bools

Logical and and

Not available in Gleam

Boolean or or ||

In Gleam both values must be bools

Logical or or

Not available in Gleam

Add + +

In Gleam both values must be ints

Add + +.

In Gleam both values must be floats

Subtract - -

In Gleam both values must be ints

Subtract - -.

In Gleam both values must be floats

Multiply * *

In Gleam both values must be ints

Multiply * *.

In Gleam both values must be floats

Divide / /

In Gleam both values must be ints

Divide / /.

In Gleam both values must be floats

Remainder % %

In Gleam both values must be ints, in Gleam negative values behave differently: Use int.modulo to mimick Python's behavior.

Concatenate + <>

In Gleam both values must be strings

Pipe |>

Gleam's pipe can pipe into anonymous functions. This operator does not exist in python

Some notes for Python: == is by default comparing by value: scalars will have their value compared the only type cast will be for 0 and 1 that will be coerced to False and True respectively variables that point to the same object will be equal with == two objects with the same members values won't be equal: no structural equality, unless the `__eq__` operator is redefined. Python operators are short-circuiting as in Gleam. Python

operators can be overloaded and be applied to any types with potential custom behaviors

Constants Python

In Python, top-level declarations are in the global/module scope is the highest possible scope. Any variables and functions defined will be accessible from anywhere in the code. There is no notion of constant variables in Python. # in the global scope `THE_ANSWER = 42`

Gleam

In Gleam constants can be created using the `const` keyword.

```
const the_answer = 42
pub fn main() { the_answer }
```

Blocks Python

Python blocks are always associated with a function / conditional / class declarations...

There is no way to create multi-line expressions blocks like in Gleam. Blocks are declared via indentation.

```
def a_func(): # A block here
    pass
```

Gleam

In Gleam braces `{ }` are used to group expressions.

```
pub fn main() {
  let x = { some_function(1) 2 }
  let y = x * {x + 10} // braces are used to change arithmetic operations order
  y }
```

Data types Strings

In Python, strings are stored as unicode code-points sequence. Strings can be encoded or decoded to/from a specific encoding.

In Gleam all strings are UTF-8 encoded binaries.

```
Python "Hellø, world!"
Gleam "Hellø, world!"
```

Tuples

Tuples are very useful in Gleam as they're the only collection data type that allows mixed types in the collection.

Python Python tuples are immutable, fixed-size lists that can contain mixed value types. Unpacking can be used to bind a name to a specific value of the tuple.

```
my_tuple = ("username", "password", 10)
_, password, _ = my_tuple
```

Gleam

```
let my_tuple = #("username", "password", 10)
let #(_, password, _) = my_tuple
```

Lists

Lists in Python are allowed to have values of mixed types, but not in Gleam.

Python Python can emulate the `cons` operator of Gleam using the `*` operator and unpacking:

```
list = [2, 3, 4]
[head, *tail] = list
# head == 2
# tail == [3, 4]
```

Gleam

Gleam has a `cons` operator that works for lists destructuring and pattern matching. In Gleam lists are immutable so adding and removing elements from the start of a list is highly efficient.

```
let list = [2, 3, 4]
let list = [1, ..list]
let [1, second_element, ..] = list
[1.0, ..list] // compile error, type mismatch
```

Dictionaries

In Python, dictionaries can have keys of any type as long as: the key type is hashable, such as integers, strings, tuples (due to their immutable values), functions... and custom mutable objects implementing the `__hash__` method. the key is unique in the dictionary. and values of any type.

In Gleam, dicts can have keys and values of any type, but all keys must be of the same type in a given dict and all values must be of the same type in a given dict. There is no dict literal syntax in Gleam, and you cannot pattern match on a dict. Dicts are generally not used much in

Gleam, custom types are more common. Python

```

{"key1": "value1", "key2": "value2"}
{"key1": "1", "key2": 2}
Gleam
import gleam/dict
dict.from_list([#("key1", "value1"), #("key2", "value2")])
dict.from_list([#("key1", "value1"), #("key2", 2)]) // Type error!

```

Flow control

Case Case is one of the most used control flows in Gleam. It can be seen as a switch statement on steroids. It provides a terse way to match a value type to an expression. Gleam's case expression is fairly similar to Python's match statement.

Matching on primitive types:

```

def http_error(status):
  match status:
    case 400: return "Bad request"
    case 404: return "Not found"
    case 418: return "I'm a teapot"

```

Matching on tuples with variable capturing:

```

match point:
  case (0, 0): print("Origin")
  case (0, y): print(f"Y={y}")
  case (x, 0): print(f"X={x}")
  case (x, y): print(f"X={x}, Y={y}")
  case _: raise ValueError("Not a point")

```

Matching on type constructors:

```

match point:
  case Point(x=0, y=0): print("Origin is the point's location.")
  case Point(x=0, y=y): print(f"Y={y} and the point is on the y-axis.")
  case Point(x=x, y=0): print(f"X={x} and the point is on the x-axis.")
  case Point(): print("The point is located somewhere else on the plane.")
  case _: print("Not a point")

```

The match expression supports guards, similar to Gleam:

```

match point:
  case Point(x, y) if x == y: print(f"The point is located on the diagonal Y=X at {x}.")
  case Point(x, y): print(f"Point is not on the diagonal.")

```

Gleam The case operator is a top level construct in Gleam:

```

case some_number {
  0 -> "Zero"
  1 -> "One"
  2 -> "Two"
  n -> "Some other number" // This matches anything
}

```

The case operator especially coupled with destructuring to provide native pattern matching:

```

case xs {
  [] -> "This list is empty"
  [a] -> "This list has 1 element"
  [a, b] -> "This list has 2 elements"
  _other -> "This list has more than 2 elements"
}

```

The case operator supports guards:

```

case xs { [a, b, c] if a > b && a <= c -> "ok"
  _other -> "ko" }

```

and disjoint union matching:

```

case number {
  2 | 4 | 6 | 8 -> "This is an even number"
  1 | 3 | 5 | 7 -> "This is an odd number"
  _ -> "I'm not sure"
}

```

Try Error management is approached differently in Python and Gleam. Python uses the notion of exceptions to interrupt the current code flow and pop up the error to the caller. An exception is raised using the keyword `raise`.

```

def a_function_that_fails():
  raise Exception("an error")

```

The callee block will be able to capture any exception raised in the block using a `try/except` set of blocks:

```

try:
  print("executed")
  a_function_that_fails()
except Exception as e:
  print("doing something with the exception", e)

```

Gleam In contrast in Gleam, errors are just containers with an associated value. A

common container to model an operation result is `Result(ReturnType, ErrorType)`. A `Result` is either: an `Error(ErrorValue)` or an `Ok(Data)` record

Handling errors actually means to match the return value against those two scenarios, using a case for instance: `case int.parse("123") { Error(e) -> io.println("That wasn't an Int") Ok(i) -> io.println("We parsed the Int") }` In order to simplify this construct, we can use the `use` expression with the `try` function from the `gleam/result` module. bind a value to the providing name if `Ok(Something)` is matched interrupt the flow and return `Error(Something)` let `a_number = "1"` let `an_error = "ouch"` let `another_number = "3"` use `int_a_number <- try(parse_int(a_number))` use `attempt_int <- try(parse_int(an_error))` // Error will be returned use `int_another_number <- try(parse_int(another_number))` // never gets executed `Ok(int_a_number + attempt_int + int_another_number)` // never gets executed

Type aliases

Type aliases allow for easy referencing of arbitrary complex types. Even though their type systems does not serve the same function, both Python and Gleam provide this feature.

Python A simple variable can store the result of a compound set of types. type `Headers = list[tuple[str, str]]` # can now be used to annotate a variable `headers: Headers = [("Content-Type", "application/json")]`

Gleam The type keyword can be used to create aliases: `pub type Headers = List(#(String, String))` let `headers: Headers = [#"Content-Type", "application/json"]`

Custom types Records Custom type allows you to define a collection data type with a fixed number of named fields, and the values in those fields can be of differing types.

Python Python uses classes to define user-defined, record-like types. Properties are defined as class members and initial values are generally set in the constructor. By default the constructor does not provide base initializers in the constructor so some boilerplate is needed: `class Person: name: str age: int def __init__(name: str, age: int) -> None: self.name = name self.age = age` `person = Person(name="Jake", age=20)` # or with positional arguments `Person("Jake", 20)` `name = person.name`

More recent alternatives are to use `dataclasses` or to leverage the `NamedTuple` base type to generate a constructor with initializers. By default a class created with the `dataclass` decorator is mutable (although you can pass options to the `dataclass` decorator to change the behavior): `from dataclasses import dataclasses @dataclass class Person: name: str age: int` `person = Person(name="Jake", age=20)` `name = person.name` `person.name = "John"` # The name is now "John"

`NamedTuples` on the other hand are immutable:

from typing import NamedTuple class Person(NamedTuple): name: str age: int person = Person(name="Jake", age=20) name = person.name # cannot reassign a value person.name = "John" # error

Gleam Gleam's custom types can be used in much the same way that structs are used in Elixir. At runtime, they have a tuple representation and are compatible with Erlang records.

```
type Person { Person(name: String, age: Int) } let person = Person(name: "Jake", age: 35) let name = person.name
```

An important difference to note is there is no OOP in Gleam. Methods can not be added to types.

Unions In Python unions can be declared with the | operator. In Gleam functions must always take and receive one type. To have a union of two different types they must be wrapped in a new custom type.

```
Python def int_or_float(x: int | float) -> str: if isinstance(x, int): return f"It's an integer: {x}" else: return f"It's a float: {x}"
```

```
Gleam type IntOrFloat { AnInt(Int) AFloat(Float) } fn int_or_float(x) { case x { AnInt(1) -> "It's an integer: 1" AFloat(1.0) -> "It's a float: 1.0" } }
```

Opaque custom types In Python, constructors cannot be marked as private. Opaque types can be imperfectly emulated using a class method and some magic property that only updates via the class factory method. In Gleam, custom types can be defined as being opaque, which causes the constructors for the custom type not to be exported from the module. Without any constructors to import other modules can only interact with opaque types using the intended API.

```
Python class OnlyCreatable: __create_key = object() @classmethod def create(cls, value): return OnlyCreatable(cls.__create_key, value) def __init__(self, create_key, value): assert(create_key == OnlyCreatable.__create_key), \ "OnlyCreatable objects must be created using OnlyCreatable.create" self.value = value
```

```
Gleam pub opaque type Identifier { Identifier(Int) } pub fn get_id() { Identifier(100) }
```

Modules Python There is no special syntax to define modules as files are modules in Python. Gleam's file is a module and named by the file name (and its directory path). Since there is no special syntax to create a module, there can be only one module in a file.

```
// in file foo.gleam pub fn identity(x) { x } // in file main.gleam import foo // if foo was in a folder called `lib` the import would be `lib/foo` pub fn main() { foo.identity(1) }
```

Imports Python # inside module src/nasa/moon_base.py # imports module src/nasa/rocket_ship.py from nasa import rocket_ship def explore_space(): rocket_ship.launch()

Gleam Imports are relative to the root src folder. Modules in the same directory will need to reference the entire path from src for the target module, even if the target

```
module is in the same folder. // inside module src/nasa/moon_base.gleam //
imports module src/nasa/rocket_ship.gleam import nasa/rocket_ship pub fn
explore_space() { rocket_ship.launch() } Named imports Python import
unix.cat as kitty Gleam import unix/cat as kitty Unqualified imports Python
from animal.cat import Cat, stroke def main(): kitty = Cat(name="Nubi")
stroke(kitty) Gleam import animal/cat.{Cat, stroke} pub fn main() { let kitty =
Cat(name: "Nubi") stroke(kitty) }
```